



Chapter 1

Introduction to Python Programming

By

Hyung Jae (Chris) Chang

Troy University



Ways to run Python Code

- Installation of Python and related packages
 - Anaconda: a complete distribution framework that includes the Python interpreter, package manager and the commonly used packages in scientific computing.
 - Miniconda: a “light” version of Anaconda that does not include the commonly used packages. You need to install all the different packages yourself, but it does include the Python interpreter and package manager.
- Ways to run Python code
 - Python shell or Ipython shell (which stands for Interactive Python)
 - Run a Python script/file from the command line (.py)
 - Use browser-based Python environment which is called Jupyter Notebook
 - Use IDEs of your choice, such as Spyder or PyCharm.



Python as Calculator

- Arithmetic operation calculated and returning answer

+	addition
-	subtraction and negation (unary)
*	multiplication
/	division (floating point)
//	division (integer)
%	modulus (remainder)
**	exponentiation

- Open up Python console and try the followings
 - 1+2
 - 5-4
 - 29*23
 - 15/3
 - 2**10
 - How about $(3 \times 4) / (2^2 + 4/2)$?



Python as Calculator (cont.)

- Let's try more!
- $(55^2 + 2/3)/(13^2)$
 - The computations are following the mathematical precedence.
 - Parenthesis can be used to change the order of operation.
- “_” is used to break the complex computations.
 - Please try the followings in the Command Prompt
 - 55^{**2}
 - $_{+2/3}$
 - $_{/13^{**2}}$

<code>**</code>	highest
<code>- (unary)</code>	•
<code>*, /, //, %</code>	•
<code>+, -</code>	lowest



Math Module

- Python has many basic arithmetic functions like sin, cos, tan, asin, acos, atan, exp, log, log10 and sqrt.
- These are used by importing “math” module
 - Type “import math” to import math module.
 - `math.sqrt(4)`: square root of 4 → square root function from math module
 - `math.pi`: pi value in math → 3.14259265359...
 - `math.sin(math.pi/2)`: sin function

TRY IT! Compute $e^{\log 10}$.

```
In [9]: math.exp(math.log(10))  
Out[9]: 10.000000000000002
```

- What do you see? We will discuss about it later.



Math Module (cont.)

- ? in Python
 - Shortcut for help
 - Try “math.factorial?”
- ZeroDivisionError
 - Python raises such an error when the expression $1/0$ (which is infinity) appears.

TRY IT! $1/0$.

```
In [12]: 1/0
```

```
-----  
ZeroDivisionError          Traceback (most recent call last)  
<ipython-input-12-9e1622b385b6> in <module>()  
----> 1 1/0
```

```
ZeroDivisionError: division by zero
```



Math Module (cont.)

- Infinite number $\rightarrow$ `math.inf`

TRY IT! Type `1/∞`, and `∞ * 2` to verify that Python handles infinity as you would expect.

```
In [13]: 1/math.inf
```

```
Out[13]: 0.0
```

```
In [14]: math.inf * 2
```

```
Out[14]: inf
```

TRY IT! Compute ∞/∞ .

```
In [15]: math.inf/math.inf
```

```
Out[15]: nan
```



Math Module (cont.)

- Complex numbers can be represented
 - A complex number: $c=a+bi$ with real part a , imaginary part b and imaginary unit $i=\sqrt{-1}$.
 - In Python, the symbol j is used to denote the imaginary unit.
 $2+5i \rightarrow 2+5j$
 - Or we can use a constructor `complex`
`complex(2, 5)`
- Scientific notation using the letter e
 - Python can handle scientific notation
 - $1e6 = 1000000$
 - $1e-3 = 0.001$



Math Module (cont.)

- Every time a function in math module is typed, it is always typed *math.function_name*.
- Alternatively, there is a simpler way. For example, if we want to use *sin* and *log* from math module, we can import them as follows: “*from math import sin, log*”.



Data Types

- In Python, there are additional data types needed for numerical values: `int`, `float` and `complex`.

- **int**: Integers, such as 1, 2, 3, ...
- **float**: Floating-point numbers, such as 3.2, 6.4, ...
- **complex**: Complex numbers, such as $2 + 5j$, $3 + 2j$, ...

Use function `type` to check the data type for different values.

TRY IT! Find out the data type for 1234.

```
In [19]: type(1234)
Out[19]: int
```

TRY IT! Find out the data type for 3.14.

```
In [20]: type(3.14)
Out[20]: float
```

TRY IT! Find out the data type for $2 + 5j$.

```
In [21]: type(2 + 5j)
Out[21]: complex
```



Logical Expressions and Operators

- Logical Expression $\rightarrow$ True or False

Table 1.1 Comparison operators.

Operator	Description	Example	Results
>	greater than	<code>a > b</code>	True
>=	greater than or equal	<code>a >= b</code>	True
<	less than	<code>a < b</code>	False
<=	less than or equal	<code>a <= b</code>	False
!=	not equal	<code>a != b</code>	True
==	equal	<code>a == b</code>	False

Table 1.2 Logical operators.

Operator	Description	Example	Results
and	greater than	P and Q	True if both P and Q are True. False otherwise
or	greater than or equal	P or Q	True if either P or Q is True. False otherwise
not	less than	not P	True if P is False. False if P is True

- We can construct truth tables for each logical operator.